

# Spec — "Choose your API" two-section onboarding (operator request 2026-05-31)

Classification: project-specific.

## Goal

The onboarding "Choose your API" step (feature/onboarding/.../steps/ApiSelectionStep.kt) gets TWO sections:

1. **On your network** (existing) — mDNS-discovered local `Endpoint.GoApi` instances.
  2. **Cloud / remote server** (NEW):
    - a manual entry field: address + port → user adds a remote API,
    - a list of pre-installed default options; we have none yet, so ONE placeholder: `https://lava.app:7777`.
- Titles/descriptions updated for the two-section layout.
  - UX: user continues with exactly ONE chosen API (local OR cloud), one selected instance.

## Established contracts (researched 2026-05-31)

- `Endpoint` (core/models `lava.models.settings.Endpoint`):
  - data class `GoApi(host: String, port: Int = 8443)` — HTTPS host:port. **Cloud uses this.**
  - data class `Mirror(host: String)` — rutracker mirror (NOT for cloud API).
- Selection flow (OnboardingViewModel):
  - `OnboardingAction.SelectApi(endpoint) → onSelectApi: connectionService.isReachable(endpoint) → on success endpointsRepository.add(endpoint) then step = Providers; on fail apiConnectivity = Failure(reason). Cloud reuses this verbatim — no new persist/probe.`
  - `RetryApiProbe` re-probes `state.selectedApi`.

- Screen call site: OnboardingScreen.kt lines ~92-100 wires  
ApiSelectionStep(...).
- §6.R: lava.app + 7777 must NOT be source literals. Pattern = .env →  
buildConfigField (see app/build.gradle.kts lines 47-52). Add:
  - .env.example: LAVA\_DEFAULT\_CLOUD\_API=https://lava.app:7777  
(placeholder, committed).
  - app/build.gradle.kts:  
buildConfigField("String", "DEFAULT\_CLOUD\_API", "\"\$\n{env[\"LAVA\_DEFAULT\_CLOUD\_API\"]}.orEmpty()}\\\"").
  - A small parser maps https://host:port →  
Endpoint.GoApi(host, port) (strip scheme, split host:port; default  
port 8443 if absent). Lives in a testable pure function (e.g.  
CloudApiDefaults.parse).

## Implementation steps

### Stream A — core feature (SINGLE-WRITER; one coherent change)

Files (all feature/onboarding/src/main/kotlin/lava/onboarding/):

1. OnboardingState.kt: add cloud-section UI state: val  
cloudAddressInput: String = "", val cloudDefaults:  
List<Endpoint> = emptyList(), val cloudAddressError: String? =  
null.
2. OnboardingAction.kt: add data class CloudAddressChanged(val  
value: String), data object AddCloudApi (parse cloudAddressInput  
→ Endpoint.GoApi → SelectApi path), (default-option tap reuses  
existing SelectApi(endpoint)).
3. OnboardingViewModel.kt:
  - inject the default-cloud config (a provider reading  
BuildConfig.DEFAULT\_CLOUD\_API; provide via Hilt so tests can fake  
it). Populate cloudDefaults on entry (parse the configured default;  
empty list if blank).
  - handle CloudAddressChanged (reduce input + clear error).
  - handle AddCloudApi: parse input; on parse-fail set  
cloudAddressError; on success call onSelectApi(parsed).
4. steps/ApiSelectionStep.kt: render two titled sections. Section 1 =  
existing discovered list (title "On your network"). Section 2 = "Cloud /  
remote server": a text field bound to cloudAddressInput + "Add" button  
(→ AddCloudApi) + the cloudDefaults rendered as ApiRow tappable (→  
onSelect). Update headline/subtitle copy. Keep §6.Q (no LazyColumn in  
verticalScroll). Add contentDescription tags: "api-cloud-input", "api-  
cloud-add", "api-cloud-default", "api-cloud-error".

5. OnboardingScreen.kt: wire the new state + callbacks into  
    ApiSelectionStep(...).
6. OnboardingHiltModule.kt: bind the default-cloud-config provider.

## **Stream B — §6.R wiring (independent-ish; touches .env.example + app/build.gradle.kts + a parser)**

- .env.example line + app/build.gradle.kts buildConfigField +  
    CloudApiDefaults parser (pure, unit-tested).

## **Stream C — tests (after A+B compile)**

- feature/onboarding/src/test/.../OnboardingViewModelTest.kt: add  
    cases — CloudAddressChanged updates input; AddCloudApi with valid  
    https://h:port → probe→persist→Providers (real VM + fakes);  
    AddCloudApi with garbage → cloudAddressError set, no advance;  
    default-option tap → same success path. Falsifiability rehearsal  
    recorded.
- CloudApiDefaults parser unit test (host/port extraction, default port,  
    reject malformed).
- New Compose Challenge app/src/androidTest/.../  
    ChallengeNNCloudApiSelectionTest.kt: drives the cloud section on a  
    real emulator — types an address, taps Add (fake probe success) →  
    asserts advance; taps the default option → asserts advance; asserts the  
    two section titles render. KDoc FALSIFIABILITY REHEARSAL block.  
    Pattern: copy Challenge26ApiDiscoveryAndConnectivityTest.kt harness  
    (TestOnboarding Hilt fakes).

## **Stream D — build + §6.Z execute + distribute (serial, after C green)**

- §6.Y bump 1.2.34-1054 → 1.2.35-1055 (app/build.gradle.kts) + lava-api-  
    go 2.3.23 → 2.3.24 if touched.
- Auth rotation for new versionCode: fresh pepper +  
    android-1.2.35-1055 client + UUID in .env (§6.AA). NOTE Gate-4 is  
    now version-aware (commit 010c9ecc) — debug+release of 1055 share  
    one pepper.
- build\_and\_release.sh (T7-backed; .containerignore now excludes .git-  
    backup\*+releases/).
- §6.Z EXECUTE on Pixel\_8/API35 host-direct+HVF: C00 + C01 + the new  
    Cloud Challenge + Challenge26 (existing discovery still works). Real  
    attestations.
- §6.AA two-stage Firebase: --debug-only → --release-only. CHANGELOG  
    1.2.35-1055 entry + snapshot + evidence.
- Commit + push all (submodules + main) to github+gitlab; converge.

## Parallelism balance (honest)

Stream A is single-writer (5 interdependent files). B's parser + C's tests + the Challenge are the genuinely-parallel pieces once A's public shape (actions/state names) is fixed. Recommended: land A's state+action shape first, then fan out B (parser+buildConfig) and C (3 test files) to subagents.

## Channel caveat

Bash channel garbles/cancels large parallel batches on this host; the sync alias fires the push hook. Work in small single calls; route output to /tmp; verify HEAD + last-version via `git ls-remote`. Restore byte-churn before commits: `git checkout -- '*.pdf' '*.html' docs/workable_items.db constitution ; rm -rf constitution/scripts/workable-items/bin.`